

数据读取

数据读取

- 1. 快速上手
- 2. 硬件接线
- 3. 使用方法
- 4. 现象结果
- 5. 代码解释

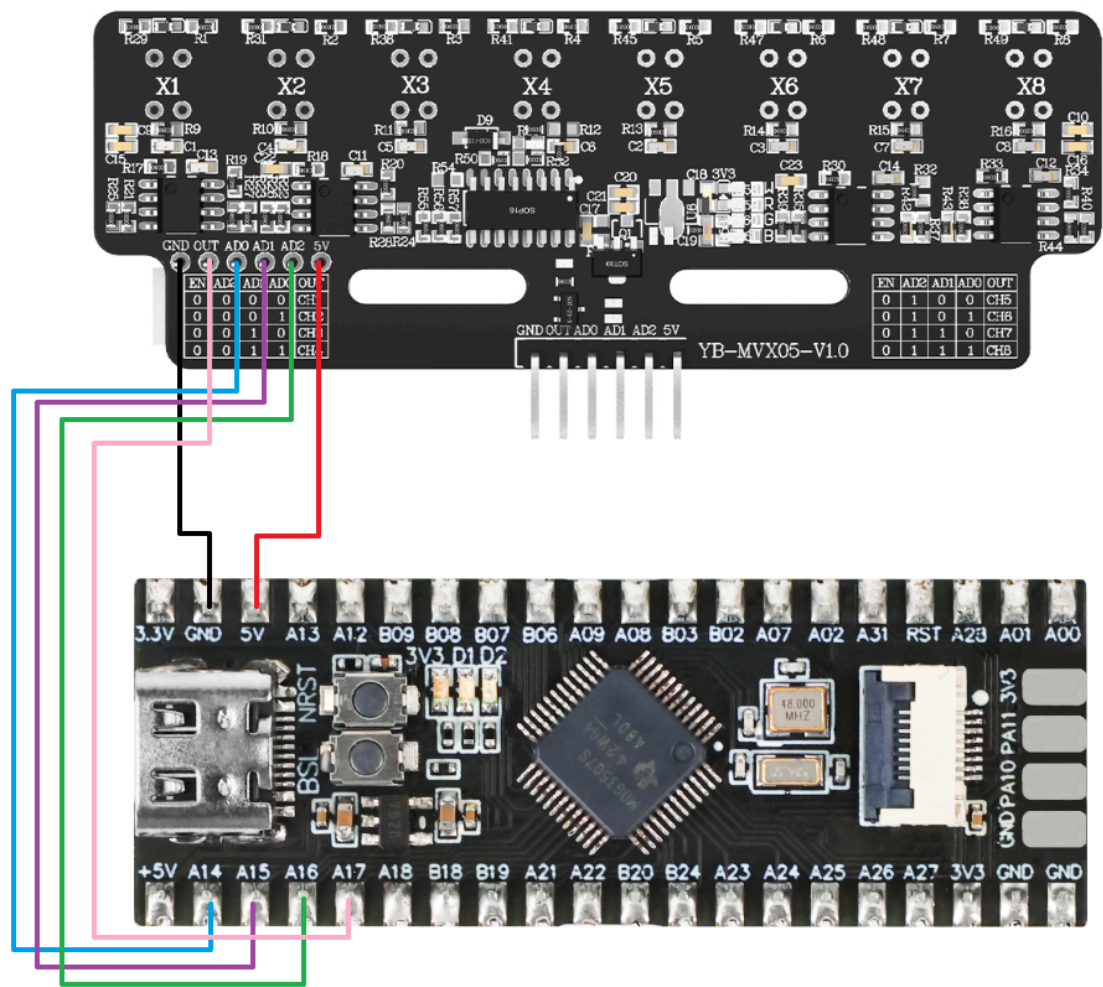
1. 快速上手

本教程讲解如何使用 MSPM0G3507主板读取八路灰度巡线模块的数字量信息，并通过串口助手查看打印结果。

教程中使用的硬件是亚博售卖的 MSPM0G3507 核心板、八路灰度巡线模块。查看下方的接线图进行接线后，再给主板烧录程序，就可以获得数据了。非亚博的模块仅供参考。

2. 硬件接线

MSPM0G3507的USB口需要使用typec与电脑USB口连接



八路灰度巡线模块	MSPM0G3507
5V	5V

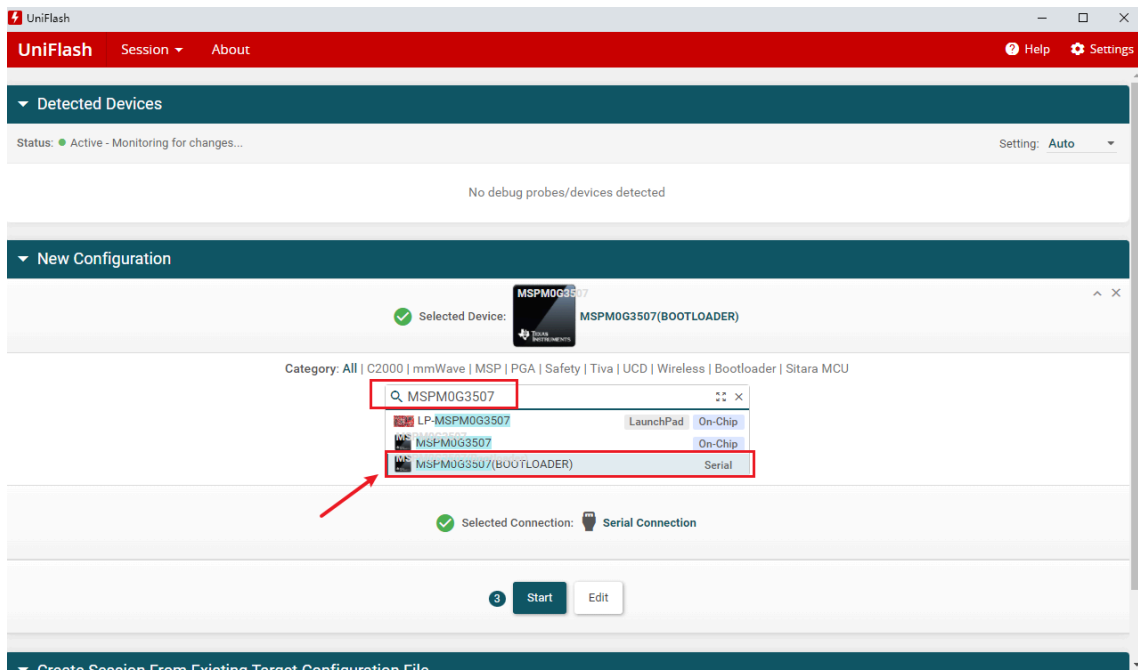
八路灰度巡线模块	MSPM0G3507
GND	GND
AD0	PA14
AD1	PA15
AD2	PA16
OUT	PA17

亚博售卖的八路灰度模块连接使用的线材为XH2.54转6pin杜邦线：



3. 使用方法

1. 打开UniFlash烧录工具，搜索MSPM0G3507，点击第三个。如果没有此软件，需要查看MSPM0G3507核心板的资料，学习主板的基础使用和烧录。



2. 在Program导航页中，选择源码里面的hex文件

Configured Device : Serial Connection > MSPM0G3507(BOOTLOADER) [download ccxml] © CORTEX_M0P

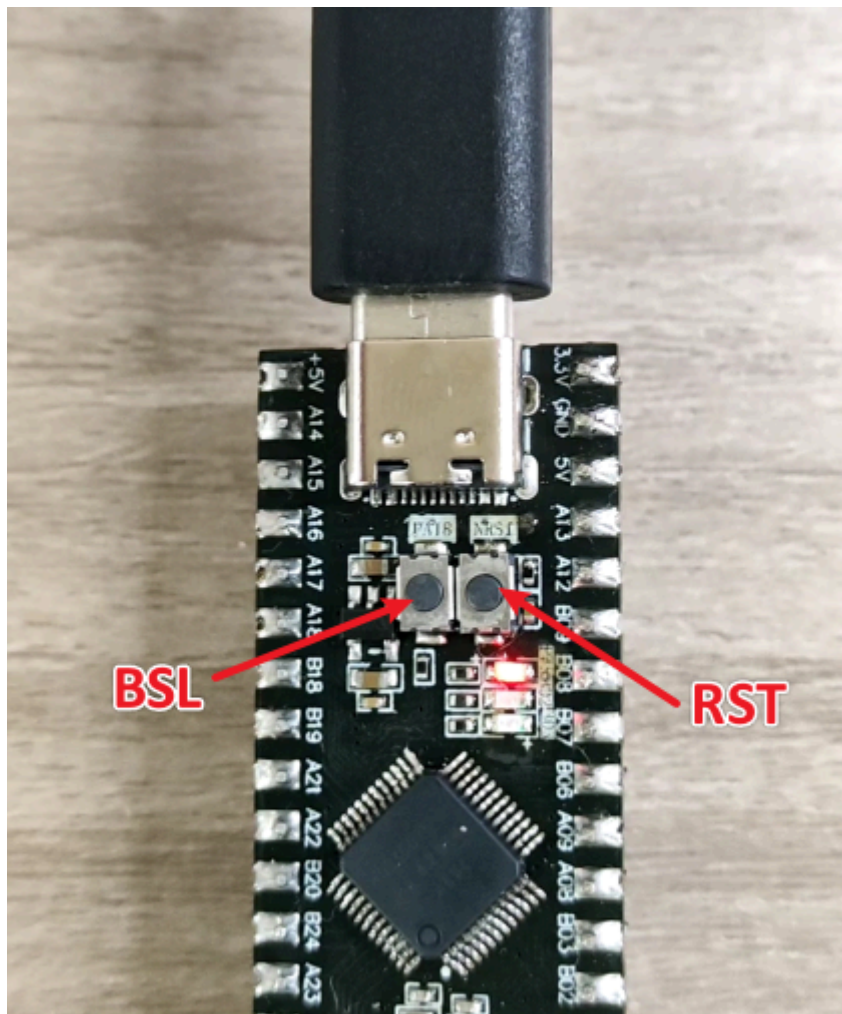
Program	Select and Load Images
Settings & Utilities	Flash Image(s) ☐ Password Browse
Standalone Command Line	☒ Application Image 1 empty.hex Size: 16.75 KB Browse X ☐ Application Image 2 Browse ☐ Application Image 3 Browse
Available Action(s) - 1 Image Selected Load Image Note: Please power cycle your device before loading images	

3. 选择COM口和波特率

Configured Device : Serial Connection > MSPM0G3507(BOOTLOADER) [download ccxml]

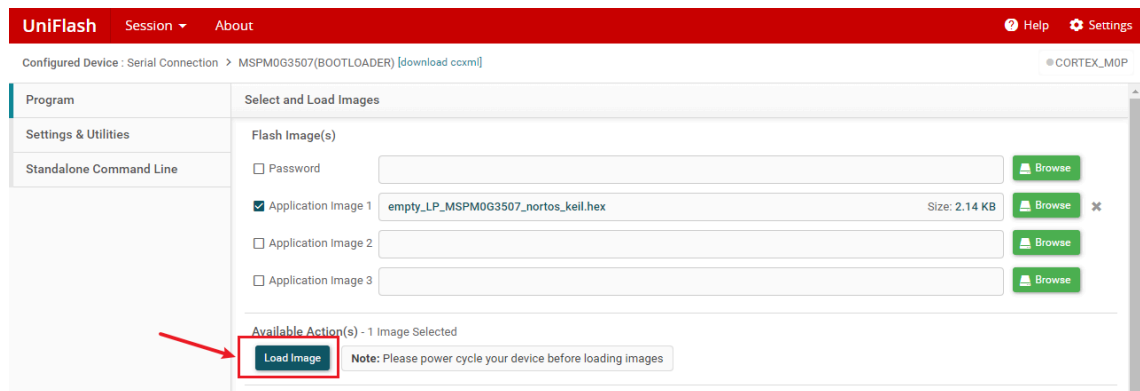
Program	Find and Configure Settings and Utilities
Settings & Utilities	Q Search: Enter Property ID Or Name To Search For Settings and But
Standalone Command Line	COM Port: COM10 ▼ Advanced Note: Only UART Communication Support is available. Communi UART Speed: ☐ 4800 ☐ 9600 ☐ 19200 ☐ 38400 ☐ 57600 ☒ 115200

4. 按住BSL按键不松手，然后再按住 RST 按键不松手，大约1S后松开 RST 按键，之后松开 BSL按键，此时就进入了烧录模式。

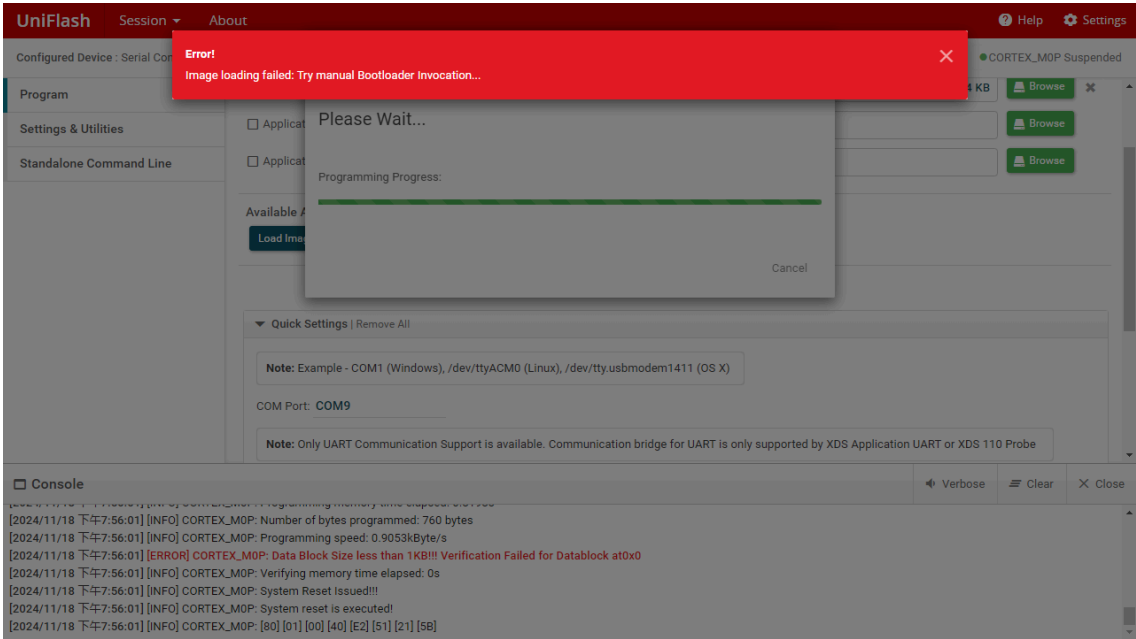


进入烧录模式后，点击 Load image 即可下载。

注意：必须在开发板进入烧录模式后的 10秒 内进行下载，否则将会下载失败。



出现如下图同样的报错 Image loading failed: Try manual Bootloader invocation... 时，其实已经下载成功了，这是下载软件的bug。



5.按下复位键NRST重启主板运行程序，并打开串口助手，查看打印的信息。

4. 现象结果

在串口助手中，您可以看到持续打印的八路灰度巡线模块的数字量。X1 对应模块上的 X1，当 X1 的灯亮起时，对应的值就为 1。

```
[2025-11-08 17:18:48.215]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.312]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.407]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.502]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.613]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.709]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]
```

5. 代码解释

```
// empty.c
int main(void)
{
    SYSCFG_DL_init();
    USART_Init();
}
```

```

while(1)
{
    Grayscale_Sensor_Read_All(g_sensor_data);

    for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
    {
        printf("[ %s:%d ]", sensor_labels[i], g_sensor_data[i]);
    }
    printf("\n");
    delay_ms(30);
}
}

```

- `main`: 程序主入口。首先调用 `SYSCFG_DL_init()`，这个函数由 TI 的 SysConfig 工具生成，负责初始化所有的硬件外设，包括 GPIO 和 UART。然后调用 `USART_Init()` 使能串口中断。在一个无限循环中，程序不断调用 `Grayscale_Sensor_Read_All()` 获取传感器数据，并通过 `printf` 打印出来。`printf` 函数已经被重定向到 UART，因此数据会通过串口发送。

```

// grayscale_sensor.c
void Grayscale_Sensor_Read_All(uint16_t* sensor_values)
{
    uint8_t i;
    for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
    {
        _select_channel(i);
        _delay_us(50);
        sensor_values[i] = Read_OUT_value();
    }
}

static void _select_channel(uint8_t channel)
{
    SENSOR_AD0_WRITE((channel >> 0) & 0x01);
    SENSOR_AD1_WRITE((channel >> 1) & 0x01);
    SENSOR_AD2_WRITE((channel >> 2) & 0x01);
}

```

- `Grayscale_Sensor_Read_All`: 遍历所有8个传感器通道，通过 `_select_channel` 选择通道，然后读取其值并存入数组。
- `_select_channel`: 通过 `SENSOR_ADx_WRITE` 宏来控制 GPIO 电平，从而选中对应的传感器。这些宏最终调用的是 TI DriverLib 库中的 `DL_GPIO_setPins` 和 `DL_GPIO_clearPins` 函数。